educación IT
CODE YOUR FUTURE

Bootcamp DevOps Engineer

Fase 1 - SysAdmin

Módulo 4

Compilación del Kernel

El kernel Linux es monolítico

Un **núcleo monolítico** es un tipo de núcleo o *kernel* de un sistema operativo. Como ejemplos de sistemas operativos de núcleo monolítico están UNIX, Linux, Solaris, DOS, AIX y FreeBSD.

Estos sistemas tienen un núcleo grande y complejo, que **engloba todos los servicios del sistema**. Está programado de forma **no modular**, y tiene un rendimiento mayor que un micronúcleo. Sin embargo, **cualquier cambio a realizar en cualquier servicio requiere la recopilación del núcleo y el reinicio del sistema** para aplicar los nuevos cambios.

Hay diversas ramificaciones de este diseño, que se han ido amoldando a nuevas necesidades. Podemos citar el **sistema de módulos ejecutables en tiempo de ejecución**, que le brinda al modelo de núcleo monolítico algunas de las ventajas de un micronúcleo. Dichos módulos pueden ser compilados, modificados, cargados y descargados en tiempo de ejecución, de manera similar a los servicios de un micronúcleo, pero con la diferencia de que se ejecutan en el espacio de memoria del núcleo mismo (anillo 0).

De esta forma, un bloqueo del módulo, es probable que bloquee todo el núcleo. Además, el módulo pasa a formar un todo con el núcleo, usando la API del mismo, y no se emplea un sistema de mensajes como en los micronúcleos. Este es el esquema usado por, entre otros, Linux, FreeBSD y varios derivados de UNIX.

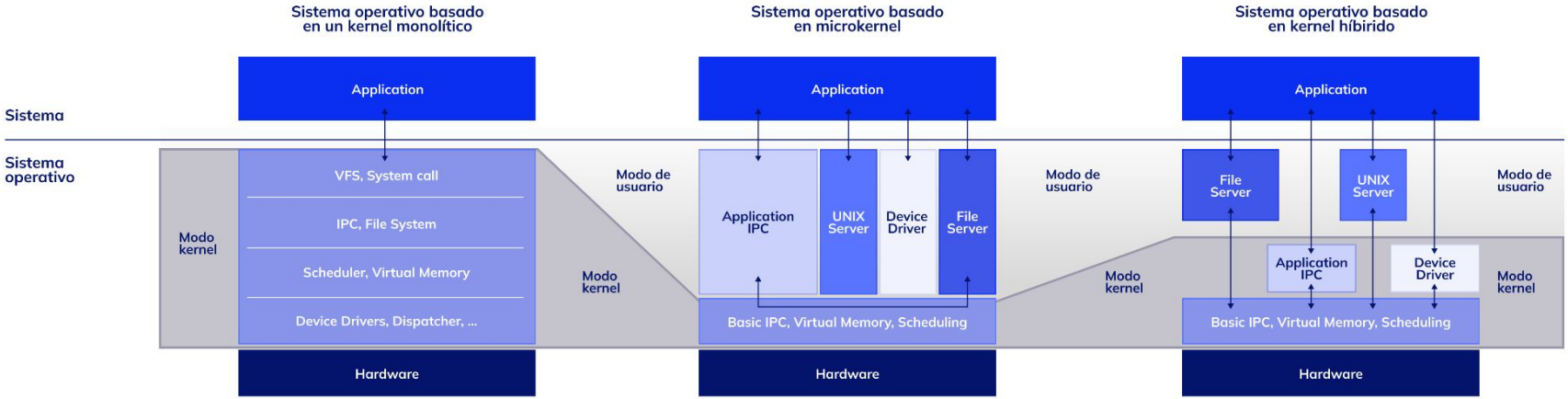
Cabe resaltar que el paso constante de mensajes entre los servicios del micronúcleo, es en parte responsable del pobre rendimiento de los micronúcleos.

Como se mencionó previamente, un sistema operativo con núcleo monolítico concentra todas las funcionalidades posibles (planificación, sistema de archivos, redes, controladores de dispositivos, gestión de memoria, etc) dentro de un gran programa. El mismo puede tener un tamaño considerable, y deberá ser recopilado por completo al añadir una nueva funcionalidad. Todos los componentes funcionales del núcleo tienen acceso a todas sus estructuras de datos internas y a sus rutinas. Un error en una rutina puede propagarse a todo el núcleo.

La alternativa es tener una **estructura de micronúcleo, donde las partes funcionales están divididas en unidades separadas** con mecanismos de comunicación estrictos entre ellos.

Arquitectura de Linux

Diagrama del núcleo



Actualmente **Linux es un núcleo monolítico híbrido**. Los controladores de dispositivos y las extensiones del núcleo normalmente se ejecutan en un espacio privilegiado conocido como *anillo 0* con acceso irrestricto al hardware, aunque algunos se ejecutan en espacio de usuario.

A diferencia de los núcleos monolíticos tradicionales, los controladores de dispositivos y las extensiones al núcleo se pueden cargar y descargar fácilmente como módulos, mientras el sistema continúa funcionando sin interrupciones. Además, los controladores pueden ser prevolcados (detenidos momentáneamente por actividades más importantes) bajo ciertas condiciones.

Esta habilidad fue agregada para gestionar correctamente interrupciones de *hardware*, y para mejorar el soporte de multiprocesamiento simétrico.

El hecho de que Linux no fuera desarrollado siguiendo el diseño de un micronúcleo (diseño que, en aquella época, era considerado el más apropiado para un núcleo por muchos teóricos informáticos) fue asunto de una famosa y acalorada discusión entre Linus Torvalds y Andrew S. Tanenbaum.

Categorías del kernel

`Prepatch o RC`	Tiene nuevas funcionalidades aún no probadas.
`Mainline`	Posee nuevas funcionalidades.
`Stable`	Tiene correcciones de errores provenientes de `Mainline`.
`Longterm`	Corrección de errores para las ramas más viejas.
`Distribution`	Cualquier versión que viene empaquetada en una distribución de Linux y no ha sido compilada “a mano” a partir del código fuente de `kernel.org`.

Compilando el kernel

Compilar un kernel consume mucho tiempo y espacio disco (al menos unos 16 GB), por otro lado en algunas distribuciones perdemos el soporte al utilizar un kernel personalizado.

Instalar paquetes necesarios para compilar

Debian	CentOS
`apt install build-essentials`	`yum group install “Development Tools”`
`apt install ncurses-dev`	`yum install binutils-devel ncurses-devel elfutils-libelf-devel hmaccalc`

Salvo que tengamos una buena razón, solo deberíamos bajar una versión '**`stable`**' o '**`longterm`**'.

> **Por ejemplo**
>
> **$ curl -O**

Otra opción es instalar el paquete de fuentes del kernel de la distribución.

> **Por ejemplo**
>
> En Debian se pueden buscar con `apt search '^linux-source'`.

Extraer las fuentes del kernel

Tradicionalmente se usaba el directorio `/usr/src` para poner las fuentes del kernel, en la actualidad eso no se recomienda y es preferible hacerlo como **usuario distinto de `root`.**

```
$ cd /home/sergio
$ tar --auto-detect -xvf linux-5.4.71.tar.xz && cd linux-5.4.71
```

Luego podemos hacer un listado de los archivos y veremos un directorio llamado **`Documentation`** el cual contiene información muy importante de los distintos componentes del kernel.

```
$ ls
arch  certs    CREDITS  Documentation  fs      init  Kbuild   kernel  LICENSES     Makefile  net     samples  security  tools  virt
block COPYING  crypto   drivers        include ipc   Kconfig  lib     MAINTAINERS  mm        README  scripts  sound     usr
```

Por ejemplo

El archivo **`Documentation/process/changes.rst`** tiene un listado de los requerimientos para instalar el kernel.

Usar la configuración del kernel actual como plantilla

Copiar la configuración del kernel actual:

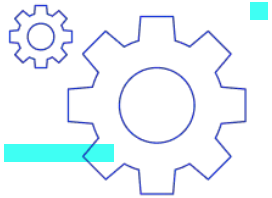
```
$ cp /boot/config-$(uname -r)  .config
```

Actualizar la configuración para adaptarlo a la nueva versión del kernel y decirle que use los valores predeterminados para las opciones nuevas:

```
$ make localmodconfig
```

Cuando se usa el archivo '`config`' de la distribución puede ser necesario hacer lo siguiente:

```
sed -i  -r 's#(CONFIG_SYSTEM_TRUSTED_KEYS=")(.*)(")#\1\3#g'  .config
```



Configuramos el kernel

Ahora vamos a personalizar nuestro kernel, para eso, deberíamos ejecutar el siguiente comando:

```
$ make menuconfig
```

Allí veremos una interfaz con menús, como la que se muestra en la siguiente pantalla.

```
.config - Linux/x86 5.4.71 Kernel Configuration
                    Linux/x86 5.4.71 Kernel Configuration
  Arrow keys navigate the menu.  <Enter> selects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.  Pressing <Y> includes,
  <N> excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Legend: [*] built-in  [ ] excluded  <M> module
  < > module capable

        *** Compiler: gcc (Debian 8.3.0-6) 8.3.0 ***
        General setup  --->
[ ] 64-bit kernel
        Processor type and features  --->
        Power management and ACPI options  --->
        Bus options (PCI etc.)  --->
        Binary Emulations  ----
        Firmware Drivers  --->
[*] Virtualization  --->
        General architecture-dependent options  --->
[*] Enable loadable module support  --->
[*] Enable the block layer  --->
        IO Schedulers  --->
        Executable file formats  --->
        Memory Management options  --->
[*] Networking support  --->
        Device Drivers  --->
        File systems  --->
        Security options  --->
-*- Cryptographic API  --->
        Library routines  --->
        Kernel hacking  --->

        <Select>    < Exit >    < Help >    < Save >    < Load >
```

Podemos navegar a través de esta interfaz con las flechas del teclado y con la tecla **TAB**. Podemos salir con **Esc** o elegir alguna de las opciones principales situadas debajo con la tecla **Enter**.

La siguiente tabla muestra los tipos de funcionalidades que podemos encontrar:

Precedidas por	Tipo de funcionalidad
`[ ]`	Pueden estar dentro o fuera de la imagen del kernel
`< >`	Pueden estar dentro de la imagen del kernel o como módulos.
`{ }`	Se seleccionan por medio de otra funcionalidad y pueden estar dentro de la imagen del kernel o como módulos
`--`	Se seleccionan por medio de otra funcionalidad.

Se puede usar la **barra de estado** para habilitar o deshabilitar funciones.

Si una funcionalidad está en blanco significa que no se compilará de ninguna manera en el kernel, si tiene '**M**' estará como módulo y si tiene '*****' estará dentro del kernel.

Una vez que terminamos nuestra configuración podemos guardarla, seleccionando **<Save>**.

Interfaces de configuración alternativas

Se pueden usar interfaces alternativas de configuración cambiando el parámetro de **`make`**, como vemos en la siguiente tabla:

Comando	Interfaz
`make`	Línea de comandos (Texto puro).
`make xconfig`	Gráfica, basada en librerías `Qt`.
`make gconfig`	Gráfica: basada en librerías `GTK+`.

Compilar

```
$ make menuconfig
```

Instalar los módulos del kernel y el kernel

```
$ su -c “make modules_install && make install”
```

El sistema se encargará de:

- Generar las dependencias entre los módulos.
- Copiar el kernel nuevo a la ubicación correcta.
- Generar el archivo de `initramfs` (disco de memoria inicial).
- Actualizar grub.

Disco de Memoria Inicial

En versiones más viejas se puede usar el comando o bien `mkinitramfs` o `dracut` para generar el archivo `initramfs`:

Uso de update-initramfs

```
# update-initramfs -u -v
```

Uso de mkinitramfs

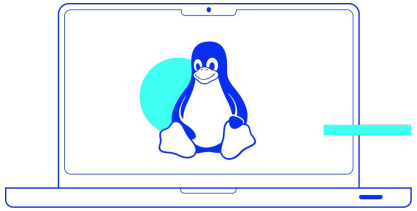
```
# mkinitramfs -k -o /boot/initrd.img-5.4.71 5.4.71
```

(El último parámetro es la versión del kernel)

Uso de dracut

En distribuciones de la rama Red Hat se utiliza `dracut`:

```
# dracut -f /boot/initramfs-5.4.71.img 5.4.71
```



Parches de actualización

Aplicar un parche para actualizar el kernel

```
$ cd linux-5.4.71
$ curl -L -O
https://cdn.kernel.org/pub/linux/kernel/v5.x/incr/patch-5.4.71-72.xz
$ xzcat patch-5.4.71-72.xz | patch -p1
```

Si se queremos revertir los cambios hacemos:

```
$ xzcat patch-5.4.71-72.xz | patch  -R -p1
```

Parches no diferenciales

Si un parche no es diferencial primero hay que aplicar en forma reversa el parche, de esa manera el kernel queda como estaba en la rama `mainline`:

```
$ curl -L -O
https://cdn.kernel.org/pub/linux/kernel/v5.x/patch-5.4.71.xz
$ xzcat patch-5.4.71.xz | patch  -R -p1
```

Una vez allí podemos aplicar el parche para pasar a la siguiente versión `stable`:

```
$ curl -L -O
https://cdn.kernel.org/pub/linux/kernel/v5.x/patch-5.4.72.xz
$ xzcat patch-5.4.72.xz | patch  -p1
```

Fuentes y recursos adicionales

- [Núcleo (informática) - Wikipedia, la enciclopedia libre]
- [Monolithic kernel - Wikipedia]
- [Building a custom kernel - Fedora Project Wiki]
- [8.10. Compilación de un núcleo]
- Capítulo 9. Trucos del sistema]
- [HowTos/Custom_Kernel - CentOS Wiki]
- [Applying Patches To The Linux Kernel — The Linux Kernel documentation]
- [Building External Modules — The Linux Kernel documentation]

educación IT
CODE YOUR FUTURE

¡Sigamos trabajando!